

# App Dev Project Report

## Placement Portal Application

---

### 1. Student Details

---

**Name:** Purujith Kadekar

**Roll Number:** 25F2008849

**Email:** [25f2008849@ds.study.iitm.ac.in](mailto:25f2008849@ds.study.iitm.ac.in)

#### About Me:

My name is Purujith Kadekar, a passionate learner, innovator, and creative individual. I am currently pursuing a dual degree — a B.S. Degree from IIT Madras (IITM BS Programme) alongside a Bachelor's in Electronics and Telecommunications Engineering at Dayananda Sagar College of Engineering, Bengaluru.

Beyond academics, I am deeply immersed in the world of classical arts. I am learning Mridanga, a classical Carnatic percussion instrument, and Yakshagana, a vibrant folk dance-drama form of Karnataka.

---

### 2. Project Details

---

**Project Title:** Placement Portal Application

#### Problem Statement

Institutes need efficient systems to manage campus recruitment activities involving companies, students, and placement drives. Current manual processes relying on spreadsheets and emails make it difficult to manage company approvals, track student applications, avoid duplicate registrations, and maintain placement records.

The goal was to build a Placement Portal Application that allows Admin (Institute), Company, and Student users to interact with the system based on their roles — enabling streamlined recruitment management from a single platform.

#### Approach

The application was built using Flask as the backend framework with a modular structure. All routing and business logic is organized in controllers.py, with database models defined in models.py, and the application entry point in app.py. Server-side rendering is handled via Jinja2 templates, and Bootstrap is used for a clean, responsive UI.

The design follows a role-based access model where Admin, Company, and Student each have separate dashboards and access permissions enforced at the route level.

---

### 3. AI/LLM Declaration

---

The following AI tools were used to a limited extent during the development of this project:

| AI Tool                                   | Purpose                                                             | Extent of Use                                   |
|-------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------|
| VS Code Built-in AI (Copilot suggestions) | Syntax suggestions and code completion                              | Low — only for syntax, not logic                |
| Brave Leo AI                              | Selecting best architectural approach and Bootstrap class selection | Low — for approach guidance and UI styling only |

Total estimated AI/LLM usage: approximately 10–15% of the overall project.

All core implementation logic, routing decisions, database design, and feature development were done entirely independently. Final debugging and integration were performed manually.

---

### 4. Technologies and Frameworks Used

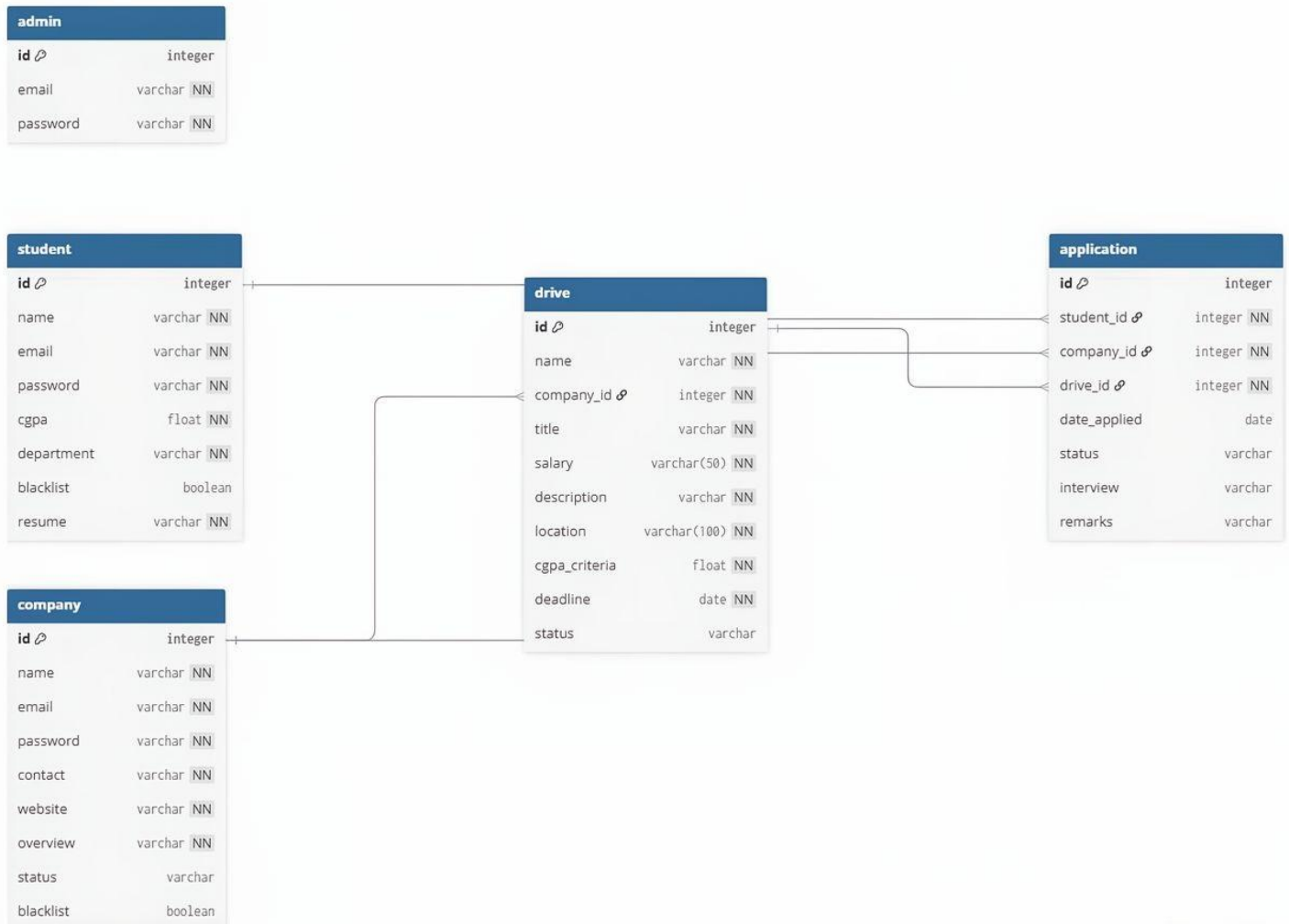
---

| Technology / Library | Purpose                                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Flask                | Core backend web framework — handles routing, request handling, and application logic                                                               |
| SQLAlchemy           | Object Relational Mapper (ORM) for SQLite — defines and manages database models                                                                     |
| SQLite               | Lightweight local database for persistent data storage                                                                                              |
| Jinja2               | Server-side templating engine for rendering dynamic HTML pages                                                                                      |
| Bootstrap 5          | Frontend CSS framework for responsive and clean UI design                                                                                           |
| Werkzeug             | Password hashing via <code>generate_password_hash</code> / <code>check_password_hash</code> ; Flask server-side sessions for login state management |
| HTML Forms (Native)  | Native HTML form submission with server-side validation and data handling in Flask routes                                                           |

---

## 5. Database Schema / ER Diagram

The application uses 5 tables managed via SQLAlchemy ORM on SQLite. Below is the schema:



## Relationships

- Company (1) → Drives (Many): One company can create multiple placement drives
- Student (1) → Application (Many): One student can apply to multiple drives
- Drives (1) → Application (Many): One drive can receive applications from many students
- Admin: Pre-existing single row — manages the entire platform (no self-registration)

## 6. API Resource Endpoints

---

This project is implemented as a purely template-based Flask application using Jinja2 for server-side rendering. No REST API endpoints have been implemented. All data submission and retrieval are handled via standard HTML form POST requests routed through Flask controllers.

---

## 7. Architecture and Features

---

### Project Structure

The application follows a clean modular structure:

- `app.py` — Application entry point, initializes Flask, DB, and registers routes
- `controllers.py` — All route handlers and business logic for Admin, Company, and Student roles
- `models.py` — SQLAlchemy database model definitions (Admin, Student, Company, Drives, Application)
- `templates/` — Jinja2 HTML templates for each role's dashboard and forms
- `static/` — Bootstrap CSS, custom styles, and any static assets

### Implemented Features

#### Admin Functionalities

- Pre-existing admin account (no self-registration required)
- Admin dashboard showing total students, companies, drives, and applications
- Approve or reject company registrations
- Approve placement drives created by companies (no drive rejection --- drives are either approved or closed)
- View and manage all students, companies, and applications
- Search students and companies by name (case-insensitive, using SQLAlchemy ilike)
- Search companies by name
- Blacklist or deactivate student and company accounts

#### Company Functionalities

- Company registration and profile management
- Login only after admin approval
- Company dashboard showing profile details, created drives, and applicant counts
- Create, edit, and close placement drives (after admin approval)
- View student applications for each drive
- Update application status (Shortlisted / Waiting / Selected / Rejected), set interview mode (In-Person / Online), and add remarks per applicant

### **Student Functionalities**

- Self-registration and login
- Student dashboard displaying approved placement drives and application statuses
- Apply for approved placement drives
- Prevention of duplicate applications for the same drive
- View application history and placement records
- Edit profile, update resume (PDF) and profile picture (JPG/PNG)

### **Other Core Functionalities**

- Role-based access control enforced at the route level
- Dynamic application status updates
- Only approved companies can create placement drives
- Students can only view and apply to approved drives

---

## **8. Video Presentation**

---

### **Drive Link:**

<https://drive.google.com/file/d/1Nusa8DuorGMwnroQKDpiE1Y1bhi5uju/view?usp=sharing>